

Sprint 14 Structured Notes: Accessible Dynamic JavaScript

0. What Sprint 14 Is About

Sprint 14 is about making interactive web pages usable for more people.

In earlier sprints, you learned how to:

- find HTML elements with JavaScript
- change page text and content
- add and remove elements
- listen for clicks, input, form submission, and other events
- open menus, update messages, and change visual state

Sprint 14 adds an important professional habit:

When JavaScript changes what the user sees, it should also update what assistive technology understands.

Assistive technology includes tools like screen readers, keyboard navigation, voice control, and other tools that help people use websites.

The main goal of Sprint 14 is:

Keep visual state and assistive-technology state in sync.

By the end of Sprint 14, you should understand:

- accessible dynamic content
- visual state
- accessibility state
- semantic state
- native HTML first
- ARIA basics
- `aria-expanded`
- `aria-controls`
- `aria-owns`
- `aria-live`
- `aria-selected`
- `aria-disabled`
- `aria-haspopup`
- `aria-required`
- `aria-checked`

- `hidden`
- custom controls
- keyboard access
- accessible dropdowns
- accessible status messages
- accessible tabs
- common Sprint 14 mistakes
- how to check whether your JavaScript state is accessible

Part A: Current Progress Before Sprint 14

1. What You Already Know

Sprint 14 depends on the work from earlier sprints.

Earlier Sprint	What You Learned	How It Helps in Sprint 14
Sprint 1	JavaScript can make a page respond.	Accessible pages also need to respond correctly.
Sprint 2	Values, variables, types, <code>let</code> , and <code>const</code> .	ARIA states are stored and updated as string values like <code>"true"</code> and <code>"false"</code> .
Sprint 3	Strings and text processing.	ARIA attributes and status messages are usually text strings.
Sprint 4	Booleans, conditions, and logical decisions.	You use booleans to decide whether something is open, closed, selected, or disabled.
Sprint 5	Functions and scope.	Accessible behavior is usually placed inside event handler functions.
Sprint 6	Arrays.	Tabs, menus, and lists may contain many related items.
Sprint 7	Loops.	You may loop through tabs or buttons to update their states.
Sprint 8	Objects and properties.	DOM elements have properties and attributes you can read and update.
Sprint 9	Callbacks and array methods.	Event listeners use callback functions.
Sprint 10	Sets and Maps.	You understand that JavaScript has different tools for organizing data.

Earlier Sprint	What You Learned	How It Helps in Sprint 14
Sprint 11	Browser APIs, DOM, and element selection.	You can find the elements that need accessibility updates.
Sprint 12	Creating, editing, and removing DOM content.	You can safely update page content with JavaScript.
Sprint 13	Events, timing, dialogs, and event flow.	You can update accessibility state when the user interacts with the page.

Simple summary:

Sprints 11–13 taught you how to find, change, and react to the page. Sprint 14 teaches you how to make those changes understandable to more users.

Part B: The Core Idea of Sprint 14

2. Visual State vs Accessibility State

When JavaScript changes the page, there are often two important states.

Visual State

Visual state means what the user can see.

Examples:

- A menu is open.
- A tab looks selected.
- A button looks disabled.
- A message appears on the screen.
- A section is hidden.

Accessibility State

Accessibility state means what assistive technology understands.

Examples:

- A screen reader knows a menu is expanded.
- A screen reader knows which tab is selected.
- A screen reader knows a custom control is disabled.
- A screen reader announces a changed status message.

The Main Problem

A sighted user may see that something changed, but assistive technology may not know unless you update the correct HTML or ARIA state.

Example:

A menu opens visually.

Bad code:

```
menu.hidden = false;
```

This shows the menu, but it does not tell assistive technology that the button is now expanded.

Better code:

```
menu.hidden = false;  
button.setAttribute("aria-expanded", "true");
```

Now two things happen:

1. The menu is visible.
2. The button reports that it is expanded.

Beginner rule:

When JavaScript changes the UI, update the DOM state and ARIA state together.

Part C: Accessibility and Dynamic Content

3. What Is Dynamic Content?

Dynamic content is page content that changes after the page has loaded.

Examples:

- A menu opens after a button click.
- A message changes from `Saving...` to `Saved successfully.`
- A tab panel changes when the user selects a different tab.
- A form error appears after validation.
- A dropdown expands.

- A notification appears.

Simple meaning:

Dynamic content is content that JavaScript changes while the user is using the page.

4. Why Dynamic Content Needs Accessibility

If JavaScript changes something silently, some users may miss the change.

Example:

```
<p id="status"></p>
```

```
status.textContent = "Saved successfully.";
```

A sighted user can see the message appear.

But a screen reader may not automatically announce it unless the element is set up as a live region.

Better HTML:

```
<p id="status" aria-live="polite"></p>
```

Now when JavaScript changes the text, assistive technology can announce the update.

Part D: Native HTML First

5. The Native-First Principle

Native HTML means using real HTML elements for their intended purpose.

Good examples:

```
<button>Open menu</button>  
<input required>  
<select></select>  
<a href="/profile">Profile</a>
```

Weak examples:

```
<div role="button">Open menu</div>  
<span onclick="submitForm()">Submit</span>
```

The weak examples may look clickable, but they do not automatically behave like real buttons or links.

6. Why Native HTML Is Better

Native HTML elements already include many built-in accessibility features.

A real `<button>` :

- can be focused with the keyboard
- can be activated with Enter or Space
- has button meaning built in
- is understood by assistive technology
- works better with forms and browser behavior

A `<div>` does not automatically have those features.

If you use a `<div role="button">`, you must manually add:

- keyboard focus
- keyboard activation
- correct role
- correct state
- correct disabled behavior

Beginner rule:

Use native HTML first. Use ARIA only when native HTML cannot describe the custom behavior clearly enough.

7. ARIA Does Not Replace Good HTML

ARIA means Accessible Rich Internet Applications.

ARIA can add extra meaning to custom or dynamic components.

But ARIA should not be used to hide poor HTML choices.

Bad habit:

```
<div role="button">Save</div>
```

Better:

```
<button>Save</button>
```

Beginner rule:

Use ARIA to improve meaning, not to replace correct HTML.

Part E: ARIA Basics

8. What Is ARIA?

ARIA is a set of attributes that can give assistive technology extra information about an element.

Examples:

```
<button aria-expanded="false">Menu</button>
```

```
<p aria-live="polite">Saved successfully.</p>
```

```
<button role="tab" aria-selected="true">Overview</button>
```

Simple meaning:

ARIA helps explain state, relationships, and behavior to assistive technology.

9. ARIA Values Are Usually Strings

In HTML, ARIA values are written as strings.

Example:

```
aria-expanded="true"  
aria-expanded="false"
```

In JavaScript, `getAttribute()` also returns strings.

Example:

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
```

This checks whether the string value is exactly `"true"`.

Important:

```
button.getAttribute("aria-expanded")
```

returns a string like:

```
"true"
```

or:

```
"false"
```

not the boolean values:

```
true  
false
```

So this pattern is common:

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
```

Then when setting the value, convert the boolean back to a string:

```
button.setAttribute("aria-expanded", String(!isExpanded));
```

Part F: `aria-expanded`

10. What Is `aria-expanded`?

`aria-expanded` tells assistive technology whether a control opens or closes something.

Common uses:

- dropdown menus
- accordions
- expandable sections
- disclosure buttons
- navigation menus

Example:

```
<button aria-expanded="false">Menu</button>
```

Simple meaning:

This button controls content that is currently collapsed.

When the content is open:

```
<button aria-expanded="true">Menu</button>
```

Simple meaning:

This button controls content that is currently expanded.

11. `aria-expanded="false"`

```
aria-expanded="false"
```

means:

The controlled content is closed or collapsed.

12. `aria-expanded="true"`

```
aria-expanded="true"
```

means:

The controlled content is open or expanded.

13. Basic `aria-expanded` Example

HTML:

```
<button id="menuButton" aria-expanded="false" aria-controls="menuList">
  Menu
</button>

<ul id="menuList" hidden>
  <li>Profile</li>
  <li>Settings</li>
  <li>Logout</li>
</ul>
```

JavaScript:

```
const button = document.getElementById("menuButton");
const menu = document.getElementById("menuList");

button.addEventListener("click", () => {
  const isExpanded = button.getAttribute("aria-expanded") === "true";

  button.setAttribute("aria-expanded", String(!isExpanded));
  menu.hidden = isExpanded;
});
```

14. Reading the Code Step by Step

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
```

This reads the current ARIA state.

If the button currently has:

```
aria-expanded="true"
```

then `isExpanded` becomes:

```
true
```

If the button currently has:

```
aria-expanded="false"
```

then `isExpanded` becomes:

```
false
```

Next:

```
button.setAttribute("aria-expanded", String(!isExpanded));
```

This flips the ARIA state.

If it was `true`, it becomes `false`.

If it was `false`, it becomes `true`.

Next:

```
menu.hidden = isExpanded;
```

This updates the visible/hidden state.

If the menu was expanded, hide it.

If the menu was collapsed, show it.

Beginner rule:

`aria-expanded` and the actual open/closed UI must always match.

Part G: `aria-controls`

15. What Is `aria-controls`?

`aria-controls` connects a control to the element it controls.

Example:

```
<button aria-controls="menuList">Menu</button>
<ul id="menuList"></ul>
```

Simple meaning:

This button controls the element whose ID is `menuList`.

16. `aria-controls` Does Not Create Behavior

This is important:

```
aria-controls="menuList"
```

does not open the menu.

It does not close the menu.

It does not add click behavior.

It only describes the relationship.

JavaScript still controls the behavior.

Example:

```
button.addEventListener("click", () => {
  menu.hidden = false;
});
```

Beginner rule:

`aria-controls` describes which element is controlled. JavaScript still does the actual controlling.

Part H: `aria-owns`

17. What Is `aria-owns`?

`aria-owns` tells assistive technology that one element should be treated as the owner or parent of another element.

Example:

```
<div role="listbox" aria-owns="option1 option2"></div>

<div id="option1" role="option">Small</div>
<div id="option2" role="option">Large</div>
```

Simple meaning:

Treat `option1` and `option2` as owned by this listbox.

18. When Might `aria-owns` Be Used?

Sometimes visual layout and DOM structure do not match.

For example, a popup or option list might be placed somewhere else in the DOM for styling or positioning.

`aria-owns` can tell assistive technology that the popup options logically belong to a control.

19. Beginner Warning About `aria-owns`

`aria-owns` is more advanced than `aria-controls`.

Most beginner projects do not need it.

Prefer simple DOM structure first:

```
<div>
  <button>Choose size</button>
  <ul>
    <li>Small</li>
    <li>Large</li>
  </ul>
</div>
```

Use `aria-owns` only when the logical relationship cannot be represented clearly with normal HTML structure.

Beginner rule:

Keep related elements together in the DOM when possible. Use `aria-owns` only for special custom components.

Part I: hidden

20. What Is hidden?

hidden is an HTML attribute that hides an element.

Example:

```
<ul id="menuList" hidden>
  <li>Profile</li>
</ul>
```

Simple meaning:

This element is currently hidden and should not be available to users.

21. Toggling hidden with JavaScript

```
menu.hidden = true;
```

means:

Hide the menu.

```
menu.hidden = false;
```

means:

Show the menu.

22. hidden and aria-expanded Together

For expandable content, update both:

```
button.setAttribute("aria-expanded", "true");
menu.hidden = false;
```

This means:

- the button says the content is expanded

- the menu is actually visible

When closing:

```
button.setAttribute("aria-expanded", "false");  
menu.hidden = true;
```

This means:

- the button says the content is collapsed
- the menu is actually hidden

Beginner rule:

Do not let `aria-expanded` say one thing while `hidden` does another thing.

Part J: `aria-live`

23. What Is `aria-live`?

`aria-live` tells assistive technology that an element may change and that the change may need to be announced.

Common uses:

- save messages
- loading messages
- form validation messages
- cart update messages
- search result counts
- notification messages

Example:

```
<p id="status" aria-live="polite"></p>
```

JavaScript:

```
status.textContent = "Saved successfully.";
```

Because the paragraph has `aria-live`, assistive technology can announce the changed text.

24. `aria-live="polite"`

```
aria-live="polite"
```

means:

Announce this update when it is a good time.

Use `polite` for normal status messages.

Examples:

```
<p aria-live="polite">Saved successfully.</p>
```

```
<p aria-live="polite">3 results found.</p>
```

25. `aria-live="assertive"`

```
aria-live="assertive"
```

means:

Announce this update quickly, even if it interrupts.

Use `assertive` only for urgent messages.

Examples:

- critical error
- session timeout warning
- important interruption

Beginner warning:

Do not use `assertive` for normal updates. It can interrupt the user too much.

26. Live Region Example

HTML:


```
<p id="status" aria-live="polite"></p>
<button id="saveButton">Save</button>
```

JavaScript:

```
const status = document.getElementById("status");
const saveButton = document.getElementById("saveButton");

saveButton.addEventListener("click", () => {
  status.textContent = "Saved successfully.";
});
```

What happens:

1. The user clicks Save.
2. JavaScript changes the status text.
3. The live region can announce the new message.

Beginner rule:

Use `aria-live` when JavaScript changes an important message after the page loads.

Part K: Common ARIA States for Custom Controls

27. What Is a Custom Control?

A custom control is an interactive component that you build yourself instead of using a normal HTML element.

Examples:

- custom tab interface
- custom dropdown
- custom switch
- custom checkbox
- custom listbox
- custom menu

Custom controls need extra care because the browser does not automatically know how they should behave.

Beginner rule:

Custom controls must communicate role, state, keyboard behavior, and visual behavior.

Part L: `aria-selected`

28. What Is `aria-selected`?

`aria-selected` tells assistive technology which item is selected.

Common uses:

- tabs
- listbox options
- grid cells
- selectable items

Example:

```
<button role="tab" aria-selected="true">Overview</button>  
<button role="tab" aria-selected="false">Details</button>
```

Simple meaning:

- Overview is selected.
- Details is not selected.

29. Updating `aria-selected`

When the selected tab changes, JavaScript should update all related tabs.

Active tab:

```
tab.setAttribute("aria-selected", "true");  
tab.tabIndex = 0;  
panel.hidden = false;
```

Inactive tab:

```
tab.setAttribute("aria-selected", "false");  
tab.tabIndex = -1;  
panel.hidden = true;
```

What this means:

- selected tab gets `aria-selected="true"`
- unselected tabs get `aria-selected="false"`
- selected tab can be reached normally with keyboard focus
- inactive tab panels are hidden

Beginner rule:

In a tab interface, only one tab should usually have `aria-selected="true"` at a time.

Part M: `aria-disabled`

30. What Is `aria-disabled`?

`aria-disabled` tells assistive technology that a custom control is disabled.

Example:

```
<button id="submitButton" aria-disabled="true">  
  Submit  
</button>
```

Simple meaning:

This control is currently disabled.

31. Important: `aria-disabled` Does Not Stop Clicks

This is a very important detail.

```
aria-disabled="true"
```

does not automatically stop JavaScript click behavior.

You must enforce the disabled behavior yourself.

Example:

```
const submitButton = document.getElementById("submitButton");
```

```
submitButton.addEventListener("click", () => {  
  const isDisabled = submitButton.getAttribute("aria-disabled") === "true";  
  
  if (isDisabled) {  
    return;  
  }  
  
  console.log("Submitted");  
});
```

This line checks the disabled state:

```
const isDisabled = submitButton.getAttribute("aria-disabled") === "true";
```

This stops the function if disabled:

```
if (isDisabled) {  
  return;  
}
```

Beginner rule:

`aria-disabled` describes disabled state. JavaScript must enforce disabled behavior.

32. Native Disabled Is Usually Better

For real buttons, use the native `disabled` attribute when possible.

Better:

```
<button disabled>Submit</button>
```

Use `aria-disabled` mostly when you are working with custom controls or when you need a control to remain focusable while being described as disabled.

Part N: `aria-haspopup`

33. What Is `aria-haspopup`?

`aria-haspopup` tells assistive technology that a control opens a popup-like element.

Common popup types:

- menu
- listbox
- tree
- grid
- dialog

Example:

```
<button aria-haspopup="menu" aria-expanded="false">  
  Options  
</button>
```

Simple meaning:

This button opens a menu-like popup.

34. Common `aria-haspopup` Values

Examples:

```
aria-haspopup="true"  
aria-haspopup="menu"  
aria-haspopup="listbox"  
aria-haspopup="dialog"
```

Beginner-friendly meanings:

Value	Meaning
<code>true</code>	This opens some kind of popup.
<code>menu</code>	This opens a menu.
<code>listbox</code>	This opens a selectable list.
<code>dialog</code>	This opens a dialog.

35. `aria-haspopup` Does Not Open Anything

Like other ARIA attributes, `aria-haspopup` only describes the behavior.

It does not create the popup.

It does not open the popup.

JavaScript still handles the action.

Example:

```
button.addEventListener("click", () => {  
  popup.hidden = false;  
  button.setAttribute("aria-expanded", "true");  
});
```

Beginner rule:

Use `aria-haspopup` to describe that a control opens a popup. Use JavaScript to actually open it.

Part O: `aria-required`

36. What Is `aria-required`?

`aria-required` tells assistive technology that a field is required.

Example:

```
<input aria-required="true">
```

Simple meaning:

This field must be filled in.

37. Native `required` Is Usually Better

For normal form fields, use the native `required` attribute.

Better:

```
<input required>
```

Why?

Because native `required` can connect with browser form validation.

`aria-required` only describes that a field is required. It does not automatically validate the form by itself.

38. When Might `aria-required` Be Used?

Use `aria-required` when building custom form-like controls that do not support native `required`.

Example:

```
<div role="combobox" aria-required="true">  
  Choose a country  
</div>
```

Beginner rule:

Use native `required` for normal form inputs. Use `aria-required` only when native validation is not available for a custom control.

Part P: `aria-checked`

39. What Is `aria-checked`?

`aria-checked` tells assistive technology whether a custom checkbox, radio, or switch is checked.

Common uses:

- custom checkbox
- custom radio button
- custom switch
- selectable option

Example:

```
<button role="switch" aria-checked="false" id="darkModeSwitch">  
  Dark mode  
</button>
```

Simple meaning:

This switch is currently off.

When on:

```
<button role="switch" aria-checked="true">  
  Dark mode  
</button>
```

Simple meaning:

This switch is currently on.

40. Toggling `aria-checked`

```
const switchButton = document.getElementById("darkModeSwitch");  
  
switchButton.addEventListener("click", () => {  
  const isChecked = switchButton.getAttribute("aria-checked") === "true";  
  switchButton.setAttribute("aria-checked", String(!isChecked));  
});
```

What happens:

1. JavaScript reads whether the switch is checked.
2. JavaScript flips the value.
3. The ARIA state is updated.

41. Native Checkbox Is Usually Better

Instead of building a custom switch, a native checkbox is often simpler and safer.

Better native option:

```
<label>  
  <input type="checkbox">  
  Dark mode  
</label>
```

Beginner rule:

Use native checkboxes when possible. Use `aria-checked` for custom controls that really need it.

Part Q: Keyboard Access

42. Why Keyboard Access Matters

Not all users use a mouse.

Some users navigate with:

- Tab
- Shift + Tab
- Enter
- Space
- Arrow keys
- Home
- End
- Escape

Accessible JavaScript should work with a keyboard, not only mouse clicks.

43. Native Buttons Already Support Keyboard Use

A real button works well:

```
<button>Open menu</button>
```

Users can:

- focus it with Tab
- activate it with Enter
- activate it with Space

A clickable `<div>` does not automatically do this.

Weak:

```
<div onclick="openMenu()">Open menu</div>
```

Better:

```
<button id="menuButton">Open menu</button>
```

Beginner rule:

If something is clickable, it is often a button or a link. Use the real HTML element when possible.

44. Keyboard Support for Custom Components

Custom components may need keyboard behavior.

Example: tabs often need these keys:

Key	Common Behavior
ArrowRight	Move to the next tab.
ArrowLeft	Move to the previous tab.
Home	Move to the first tab.
End	Move to the last tab.
Enter or Space	Activate a tab or control.

Beginner rule:

A custom control is not complete if it only works with mouse clicks.

Part R: Beginner Mental Model for Accessible Interactions

45. The Three Questions

For every interactive component, ask these three questions.

Question 1: What Changes Visually?

Example:

- The menu appears.
- The tab panel changes.
- The button looks disabled.
- The status text changes.

Question 2: What Changes Semantically?

Semantic state means the meaning/state exposed through HTML or ARIA.

Examples:

- `aria-expanded` changes from `"false"` to `"true"`
- `aria-selected` changes from `"false"` to `"true"`
- `aria-checked` changes from `"false"` to `"true"`
- `aria-disabled` changes from `"false"` to `"true"`

Question 3: Can the Keyboard Use It?

Examples:

- Can the user Tab to it?
- Can the user activate it with Enter or Space?
- Can the user move through tabs with arrow keys?
- Can the user close a popup with Escape if that behavior is expected?

A weak component only changes visually.

A stronger component changes:

1. visual state
2. semantic/accessibility state
3. keyboard behavior

Part 5: Mini-Build 1 — Accessible Dropdown

46. Goal

Build a dropdown menu where:

- the button opens and closes the menu
- `aria-expanded` matches the open/closed state
- `aria-controls` connects the button to the menu
- `hidden` hides and shows the menu
- `aria-live` announces status updates

47. HTML

```
<button id="menuButton" aria-expanded="false" aria-controls="menuList">
  Menu
</button>

<ul id="menuList" hidden>
```

```
<li><a href="/profile">Profile</a></li>
<li><a href="/settings">Settings</a></li>
<li><a href="/logout">Logout</a></li>
</ul>

<p id="status" aria-live="polite"></p>
```

48. JavaScript

```
const menuButton = document.getElementById("menuButton");
const menuList = document.getElementById("menuList");
const status = document.getElementById("status");

menuButton.addEventListener("click", () => {
  const isExpanded = menuButton.getAttribute("aria-expanded") === "true";
  const nextExpanded = !isExpanded;

  menuButton.setAttribute("aria-expanded", String(nextExpanded));
  menuList.hidden = !nextExpanded;

  if (nextExpanded) {
    status.textContent = "Menu opened.";
  } else {
    status.textContent = "Menu closed.";
  }
});
```

49. What This Code Does Correctly

This updates accessibility state:

```
menuButton.setAttribute("aria-expanded", String(nextExpanded));
```

This updates visual/DOM state:

```
menuList.hidden = !nextExpanded;
```

This updates a live region:

```
status.textContent = "Menu opened.";
```

This keeps the UI understandable.

Part T: Mini-Build 2 — Accessible Status Message

50. Goal

Build a button that updates a status message after an action.

51. HTML

```
<button id="saveButton">Save</button>
<p id="saveStatus" aria-live="polite"></p>
```

52. JavaScript

```
const saveButton = document.getElementById("saveButton");
const saveStatus = document.getElementById("saveStatus");

saveButton.addEventListener("click", () => {
  saveStatus.textContent = "Saved successfully.";
});
```

53. Why This Is Accessible

The message is placed inside a live region:

```
aria-live="polite"
```

So assistive technology can announce the changed message.

Beginner rule:

Important dynamic messages should not only appear visually. They should also be announced when appropriate.

Part U: Mini-Build 3 — Accessible Tabs

54. Goal

Build a simple tab interface where:

- one tab is selected
- one panel is visible
- inactive panels are hidden
- ARIA state matches the selected tab

55. HTML

```
<div role="tablist" aria-label="Account sections">
  <button
    id="tab-overview"
    role="tab"
    aria-selected="true"
    aria-controls="panel-overview"
    tabindex="0"
  >
    Overview
  </button>

  <button
    id="tab-settings"
    role="tab"
    aria-selected="false"
    aria-controls="panel-settings"
    tabindex="-1"
  >
    Settings
  </button>
</div>

<section
  id="panel-overview"
  role="tabpanel"
  aria-labelledby="tab-overview"
>
  Overview content
</section>

<section
  id="panel-settings"
```

```
    role="tabpanel"
    aria-labelledby="tab-settings"
    hidden
  >
    Settings content
</section>
```

56. Basic JavaScript

```
const overviewTab = document.getElementById("tab-overview");
const settingsTab = document.getElementById("tab-settings");
const overviewPanel = document.getElementById("panel-overview");
const settingsPanel = document.getElementById("panel-settings");

function selectOverview() {
  overviewTab.setAttribute("aria-selected", "true");
  overviewTab.tabIndex = 0;
  overviewPanel.hidden = false;

  settingsTab.setAttribute("aria-selected", "false");
  settingsTab.tabIndex = -1;
  settingsPanel.hidden = true;
}

function selectSettings() {
  settingsTab.setAttribute("aria-selected", "true");
  settingsTab.tabIndex = 0;
  settingsPanel.hidden = false;

  overviewTab.setAttribute("aria-selected", "false");
  overviewTab.tabIndex = -1;
  overviewPanel.hidden = true;
}

overviewTab.addEventListener("click", selectOverview);
settingsTab.addEventListener("click", selectSettings);
```

57. What This Code Does Correctly

For the active tab:

```
tab.setAttribute("aria-selected", "true");
tab.tabIndex = 0;
panel.hidden = false;
```

For inactive tabs:

```
tab.setAttribute("aria-selected", "false");
tab.tabIndex = -1;
panel.hidden = true;
```

This keeps:

- selected state correct
- focus behavior clearer
- panel visibility correct

Beginner rule:

Tabs must update selected state and panel visibility together.

Part V: Common Sprint 14 Mistakes

58. Mistake 1: Only Changing CSS

Weak code:

```
menu.classList.toggle("open");
```

This may visually open the menu, but it does not update ARIA state.

Better:

```
const isOpen = menu.classList.toggle("open");
button.setAttribute("aria-expanded", String(isOpen));
menu.hidden = !isOpen;
```

Why better?

- CSS class changes visual state.
- `aria-expanded` changes accessibility state.

- `hidden` changes DOM availability.

59. Mistake 2: `aria-expanded` Does Not Match the UI

Wrong situation:

```
<button aria-expanded="false">Menu</button>
```

But the menu is visible.

This is bad because the visual state and accessibility state disagree.

Correct idea:

If the menu is visible:

```
aria-expanded="true"
```

If the menu is hidden:

```
aria-expanded="false"
```

60. Mistake 3: Using ARIA Instead of Native HTML

Weak:

```
<div role="button">Submit</div>
```

Better:

```
<button>Submit</button>
```

A real button already has proper button behavior.

61. Mistake 4: Thinking `aria-disabled` Disables the Button

Wrong assumption:

```
<button aria-disabled="true">Save</button>
```

This does not automatically block clicks.

Better:

```
button.addEventListener("click", () => {  
  if (button.getAttribute("aria-disabled") === "true") {  
    return;  
  }  
  
  saveData();  
});
```

62. Mistake 5: Using `aria-live="assertive"` Too Much

Weak:

```
<p aria-live="assertive">Saved.</p>
```

For a normal save message, this is too aggressive.

Better:

```
<p aria-live="polite">Saved.</p>
```

Use `assertive` only for urgent messages.

63. Mistake 6: Creating Custom Controls Without Keyboard Support

Weak:

```
<div role="button" onclick="openMenu()">Menu</div>
```

Problems:

- may not be focusable
- may not work with Enter
- may not work with Space
- may not communicate state correctly

Better:

```
<button id="menuButton" aria-expanded="false" aria-controls="menuList">
  Menu
</button>
```

Part W: Core Code Patterns to Memorize

64. Toggle Expanded State

```
const isExpanded = button.getAttribute("aria-expanded") === "true";

button.setAttribute("aria-expanded", String(!isExpanded));
menu.hidden = isExpanded;
```

Use this for:

- dropdowns
- menus
- accordions
- expandable sections

65. Update a Live Region

```
<p id="status" aria-live="polite"></p>
```

```
status.textContent = "Saved successfully.";
```

Use this for:

- save messages
- loading messages
- result count updates
- non-urgent notifications

66. Block Disabled Custom Control Behavior

```
button.addEventListener("click", () => {
  const isDisabled = button.getAttribute("aria-disabled") === "true";
```

```
if (isDisabled) {  
    return;  
}  
  
runAction();  
});
```

Use this when:

- a custom control has `aria-disabled="true"`
- you need JavaScript to stop the action

67. Toggle Checked State

```
const isChecked = switchButton.getAttribute("aria-checked") === "true";  
switchButton.setAttribute("aria-checked", String(!isChecked));
```

Use this for:

- custom switches
- custom checkboxes
- custom toggle buttons

68. Select a Tab

```
activeTab.setAttribute("aria-selected", "true");  
activeTab.tabIndex = 0;  
activePanel.hidden = false;  
  
inactiveTab.setAttribute("aria-selected", "false");  
inactiveTab.tabIndex = -1;  
inactivePanel.hidden = true;
```

Use this for:

- tab interfaces
- custom selected item groups

Part X: Sprint 14 Debug Checklist

69. Accessible Interaction Checklist

When you build an interactive component, check these questions.

Visual State

- Does the component visually open, close, select, disable, or update correctly?
- Does the visible state match the JavaScript state?

Accessibility State

- Does `aria-expanded` match open/closed state?
- Does `aria-selected` match selected state?
- Does `aria-checked` match checked state?
- Does `aria-disabled` match disabled state?
- Does `aria-controls` point to the correct element ID?
- Does `aria-live` exist for important dynamic messages?

Keyboard Use

- Can the user reach the control with Tab?
- Can the user activate it with keyboard controls?
- Do custom tabs support expected navigation?
- Does focus go somewhere logical?

Native HTML

- Could this be a real `<button>` ?
- Could this be a real `<input>` ?
- Could this be a real `<select>` ?
- Could this be a real `<a href="...">` ?

If yes, use native HTML first.

Part Y: Sprint 14 Revision Questions

70. Concept Questions

1. What is the difference between visual state and accessibility state?
2. Why is changing CSS alone not enough for accessible dynamic components?
3. What does `aria-expanded` communicate?

4. What does `aria-controls` communicate?
5. Does `aria-controls` create behavior by itself?
6. What does `hidden` do?
7. What is `aria-live` used for?
8. What is the difference between `aria-live="polite"` and `aria-live="assertive"`?
9. Why should `assertive` be used carefully?
10. What does `aria-selected` communicate?
11. What does `aria-disabled` communicate?
12. Does `aria-disabled` automatically stop click behavior?
13. What does `aria-checked` communicate?
14. What does `aria-haspopup` communicate?
15. What does `aria-required` communicate?
16. Why is native `required` usually better than `aria-required` on normal inputs?
17. Why is a real `<button>` usually better than `<div role="button">`?
18. Why do custom controls need keyboard support?
19. What three questions should you ask for every interactive component?
20. What does it mean to keep visual state and assistive-technology state in sync?

71. Code Questions

1. What does this line do?

```
const isExpanded = button.getAttribute("aria-expanded") === "true";
```

1. What does this line do?

```
button.setAttribute("aria-expanded", String(!isExpanded));
```

1. What does this line do?

```
menu.hidden = isExpanded;
```

1. Why is this incomplete?

```
menu.classList.toggle("open");
```

1. Why is this not enough by itself?

```
<button aria-disabled="true">Save</button>
```

1. Why is this useful?

```
<p aria-live="polite"></p>
```

Part Z: Sprint 14 Passing Standard

You are ready to move on from Sprint 14 when you can build and explain this pattern from memory:

```
const expanded = button.getAttribute("aria-expanded") === "true";

button.setAttribute("aria-expanded", String(!expanded));
menu.hidden = expanded;
```

You should be able to explain:

- `getAttribute()` reads the current ARIA state.
- ARIA values from attributes are strings.
- `aria-expanded` must match whether the controlled content is open.
- `hidden` controls whether the content is currently shown or hidden.
- JavaScript must update visual state and accessibility state together.
- `aria-controls` describes a relationship but does not create behavior.
- `aria-live` is for dynamic status messages.
- `aria-disabled` describes disabled state but does not stop behavior by itself.
- `aria-selected` identifies the selected item in custom tab/listbox/grid-style controls.
- `aria-checked` identifies checked state in custom checkbox/switch/radio-style controls.
- `aria-haspopup` describes that a control opens a popup.
- `aria-required` describes that a custom field is required.
- Native HTML controls are usually better than custom ARIA controls when they solve the problem.
- Keyboard users must be able to operate interactive controls.

Final Sprint 14 summary:

Sprint 14 teaches you to make JavaScript interactions accessible by keeping what users see, what assistive technology understands, and what keyboard users can operate in sync.